Container Orchestration with Kubernetes using Minikube

Author

Oluwaseun Osunsola
LinkedIn

Project Overview

This project demonstrates the setup and deployment of a local Kubernetes environment using **Minikube** and **Docker** on Ubuntu (VMware). It covers:

- Environment preparation
- Docker installation and validation
- Minikube cluster setup
- Kubernetes CLI (kubectl) usage
- Application deployment and exposure
- Scaling and self-healing demonstration

Project Objectives

By the end of this project:

- Understand Kubernetes architecture and components
- Deploy a local Kubernetes cluster using Minikube
- Build and run containerized applications
- Perform scaling and observe self-healing

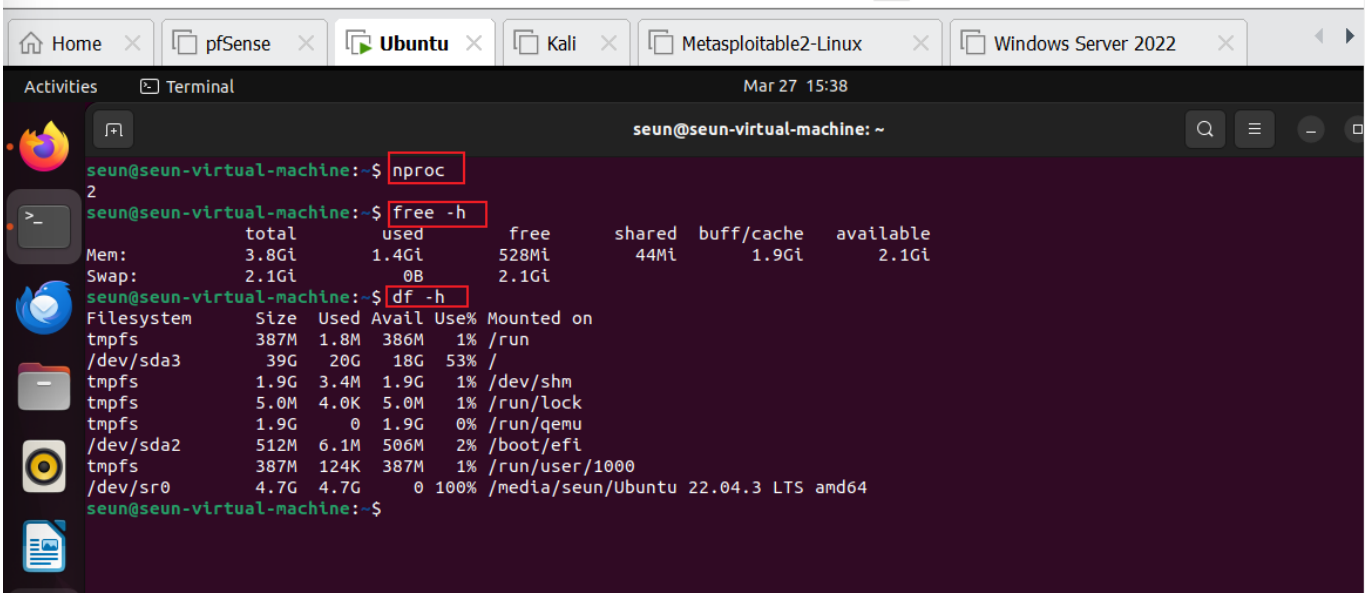
System Requirements

- CPU: ≥ 2 cores
- RAM: ≥ 2GB
- Disk: ≥ 20GB

PHASE 1: System Verification

Check available system resources:

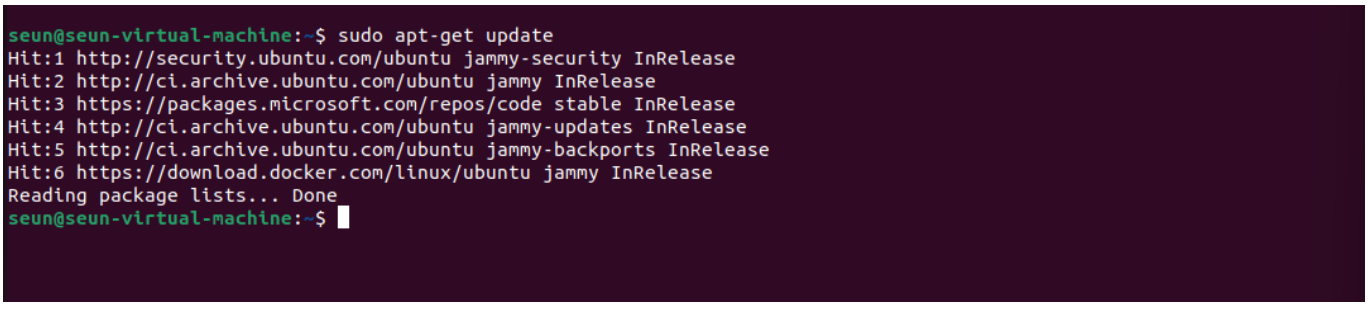
```
nproc
free -h
df -h
```



◇ PHASE 2: Install Docker

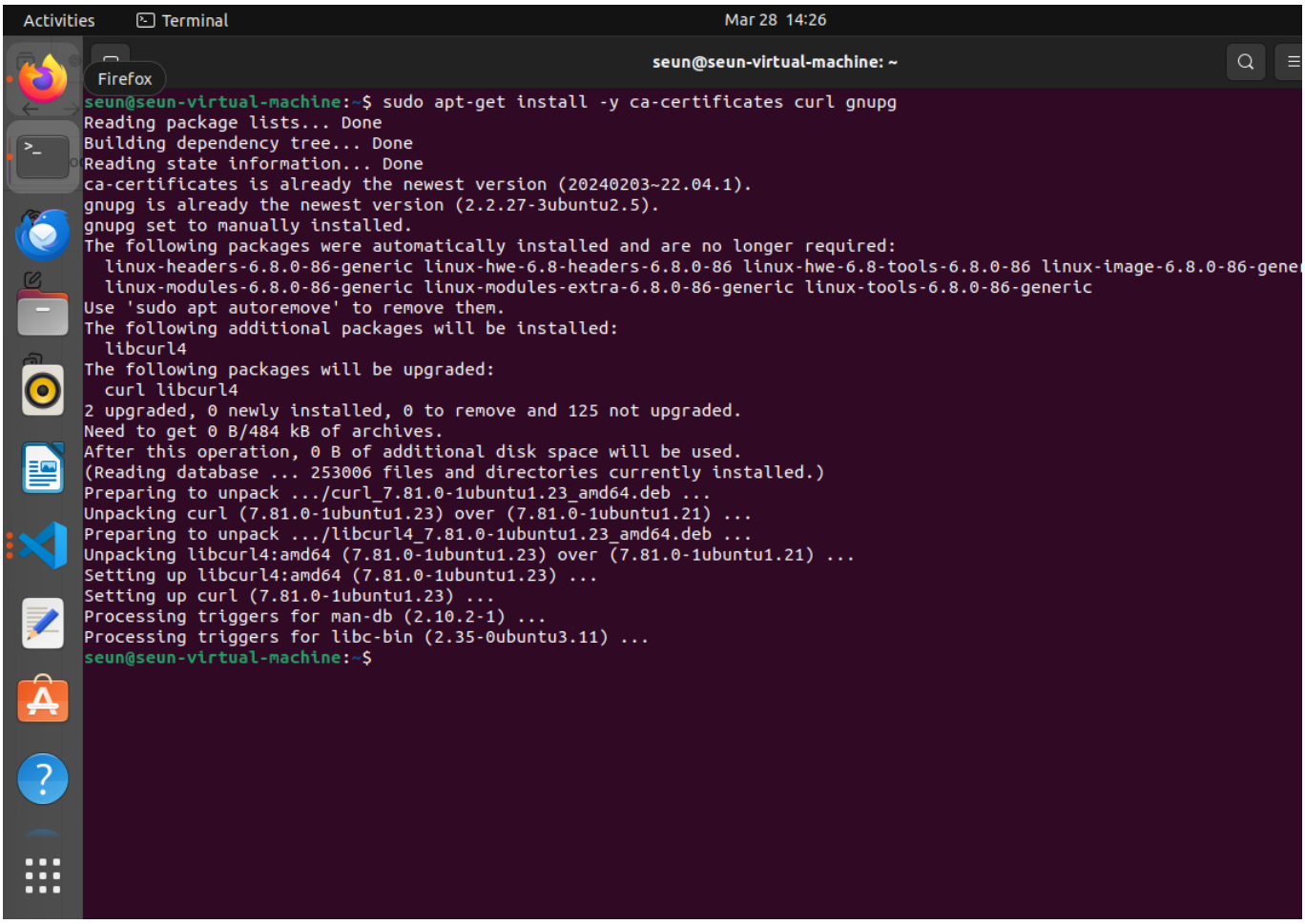
Step 1: Update package index

```
sudo apt-get update
```



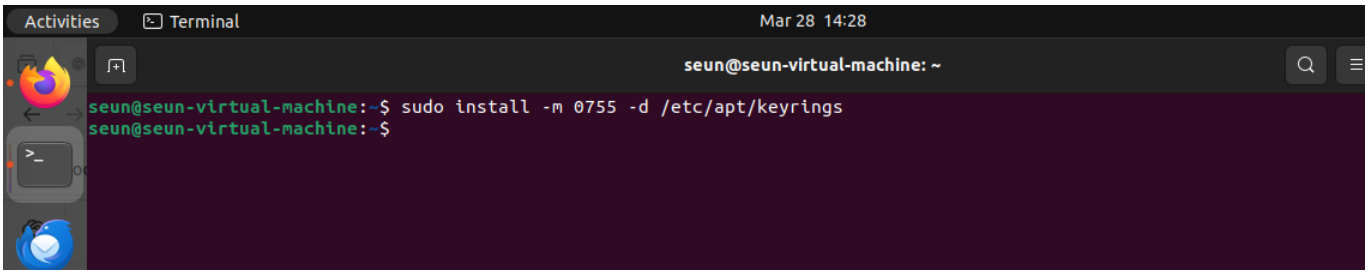
Step 2: Install required packages

```
sudo apt-get install ca-certificates curl gnupg
```



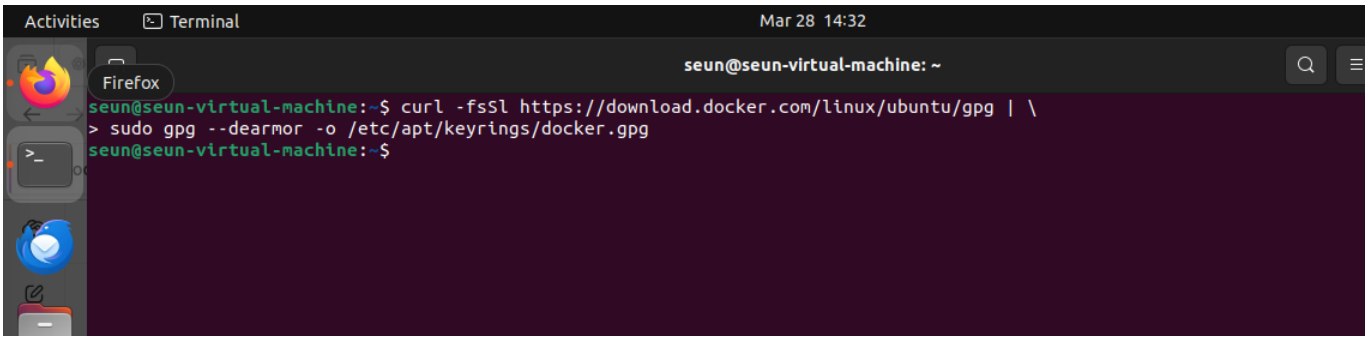
Step 3: Create keyring directory

```
sudo install -m 0755 -d /etc/apt/keyrings
```



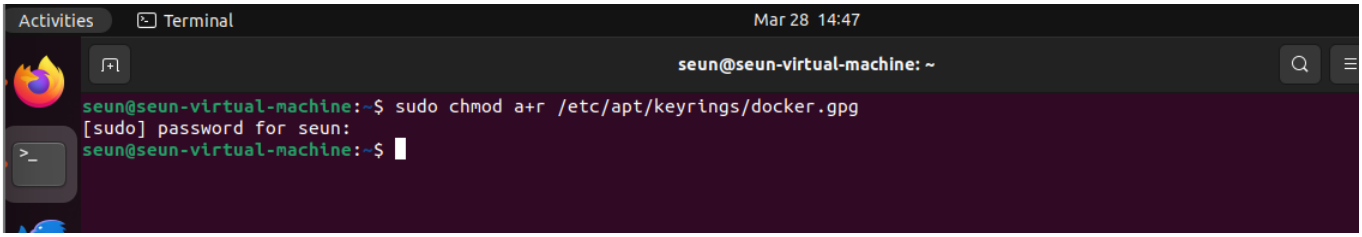
Step 4: Add Docker GPG key

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```



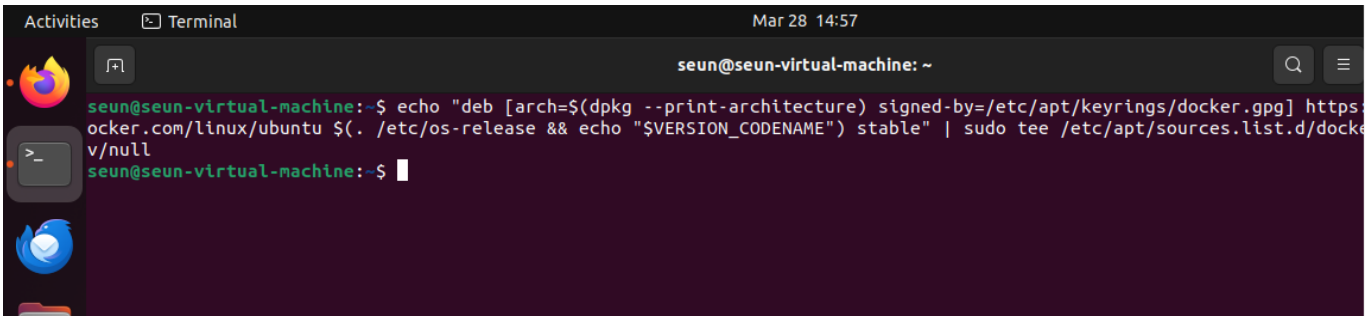
Step 5: Set permissions

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

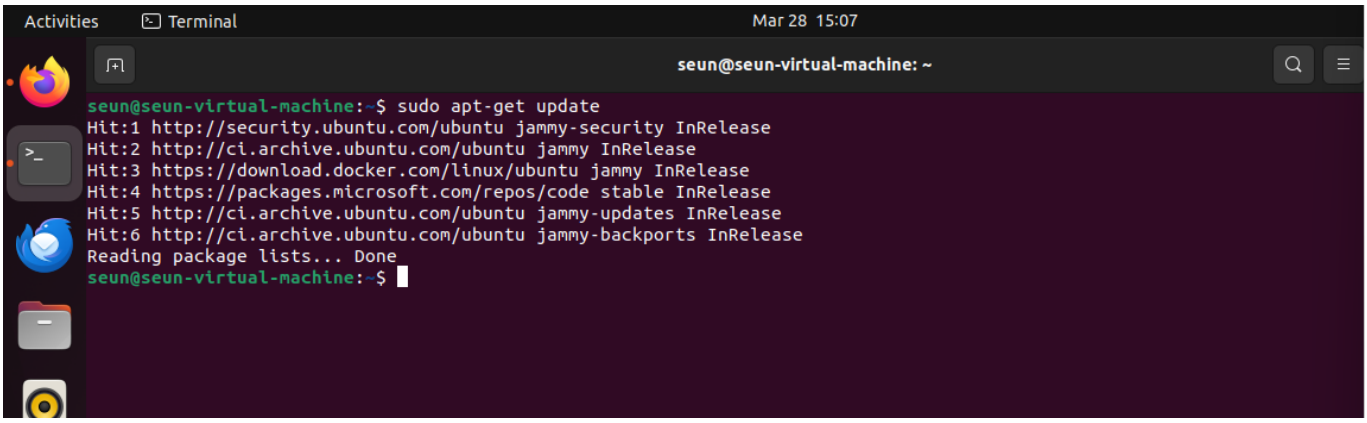


Step 6: Add Docker repository

```
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

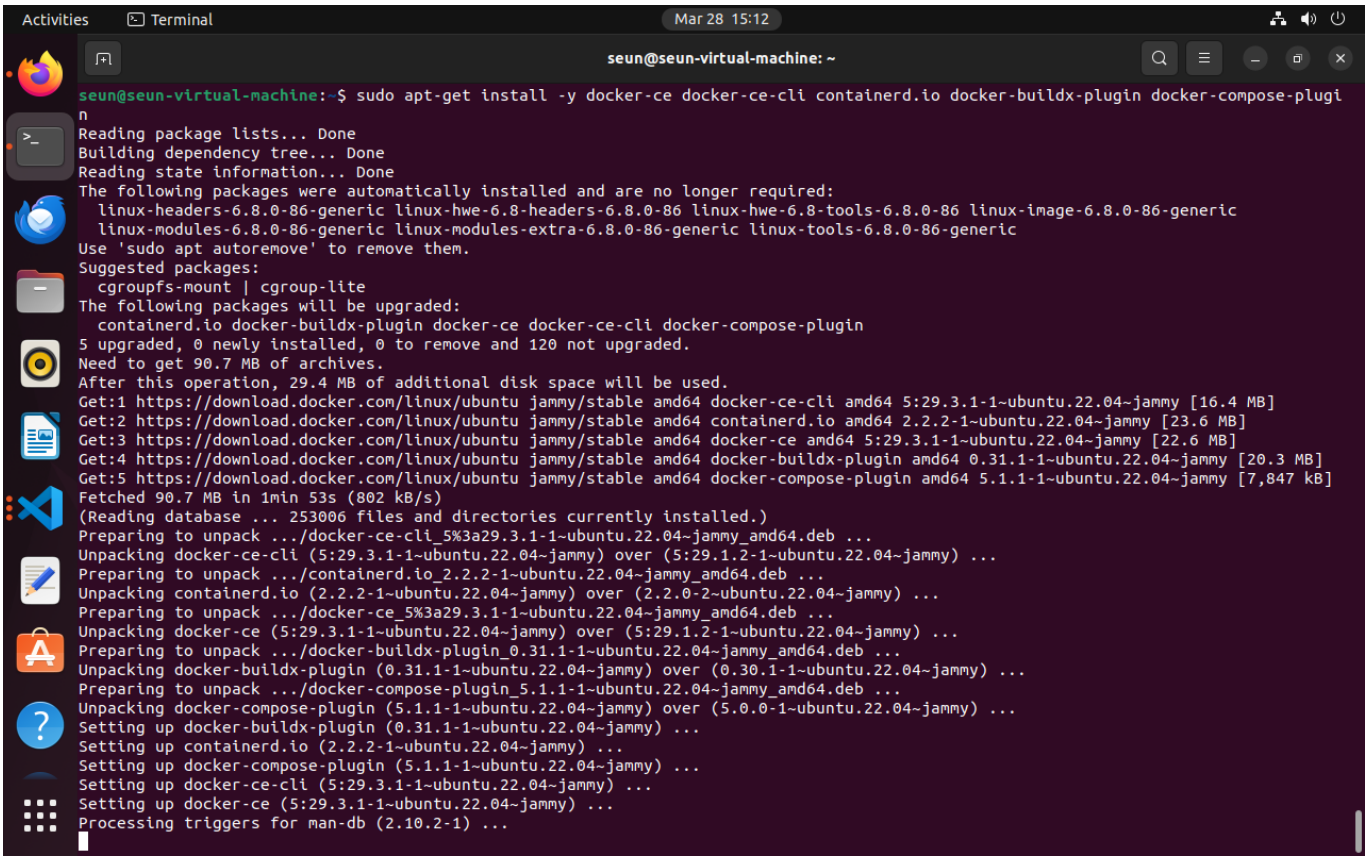


Step 7: Update package index again



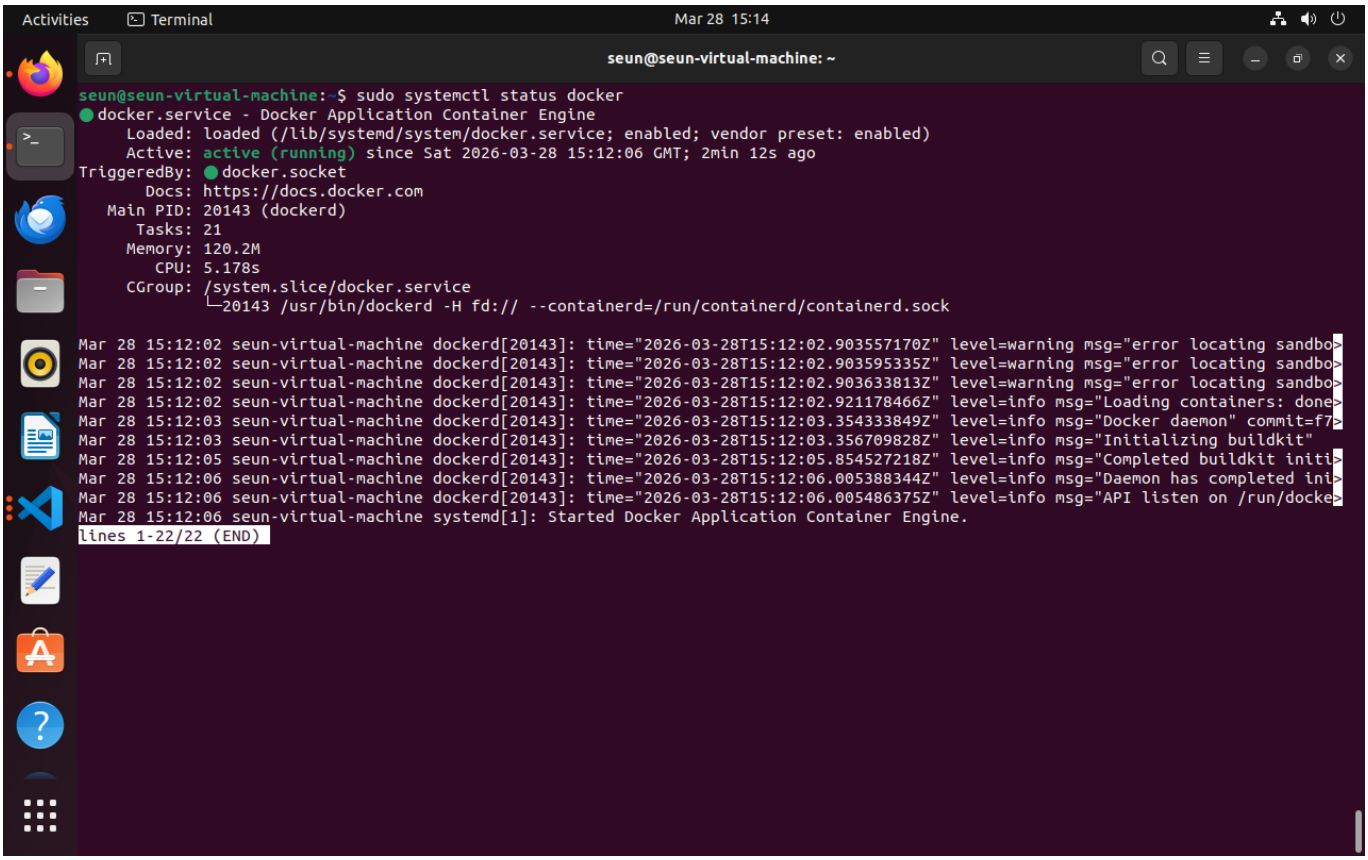
Step 8: Install Docker

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```



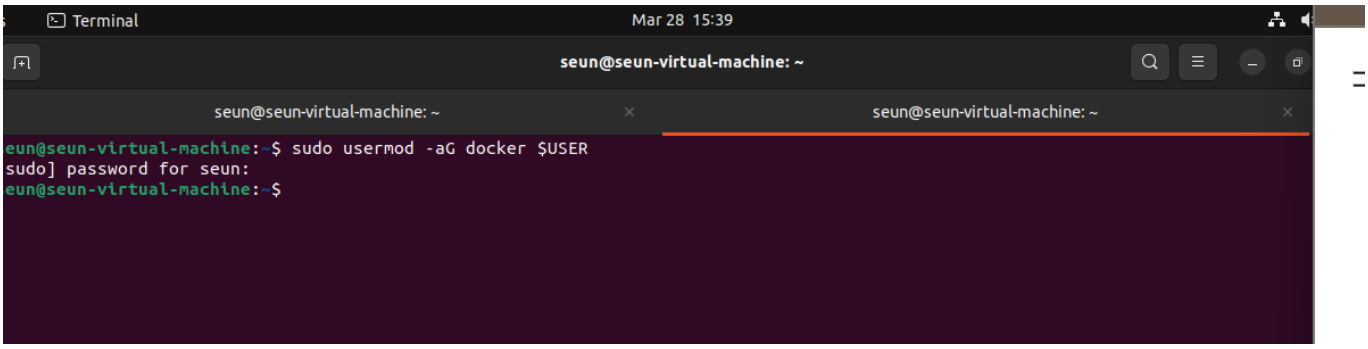
Step 9: Verify Docker

```
sudo systemctl status docker
```



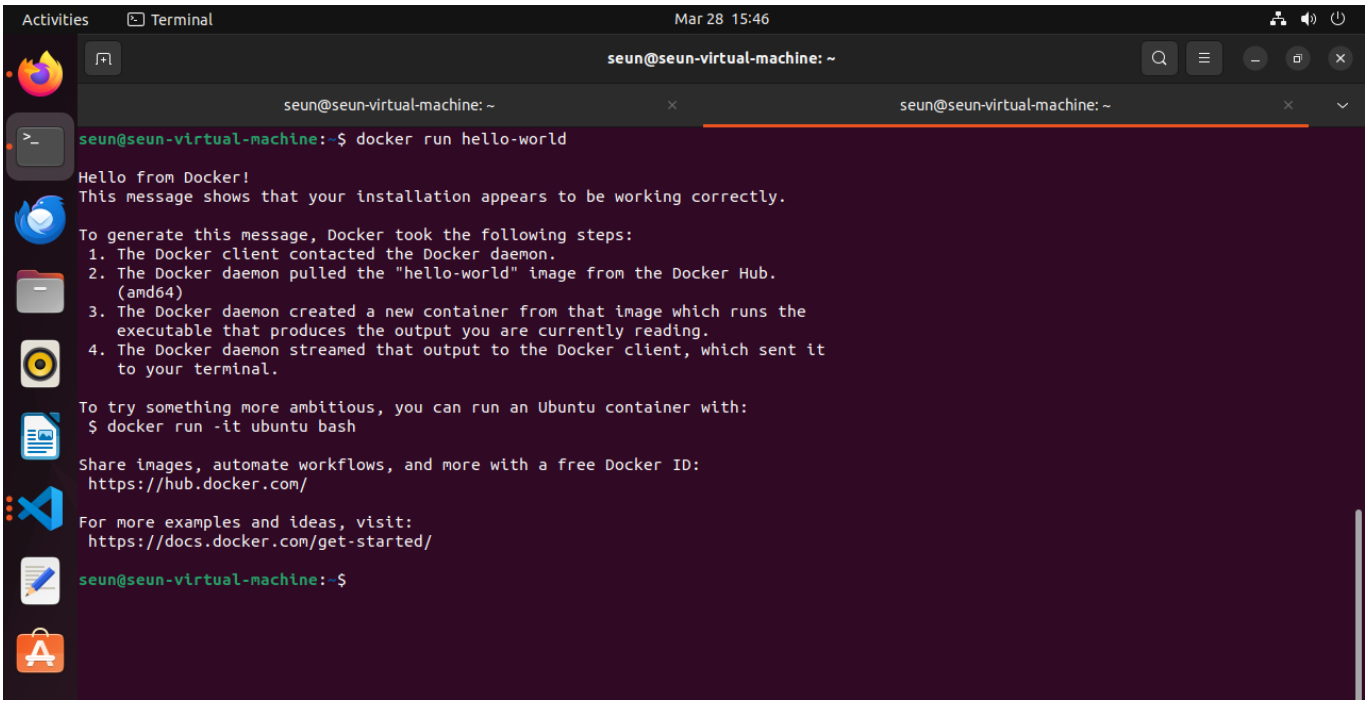
Step 10: Add user to Docker group

```
sudo usermod -aG docker $USER
```



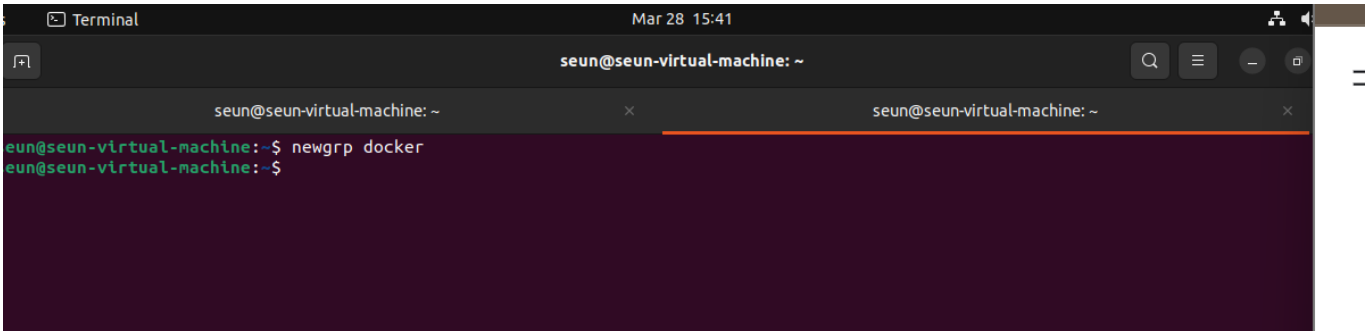
Step 11: Test Docker

```
docker run hello-world
```



Step 12: Apply group changes

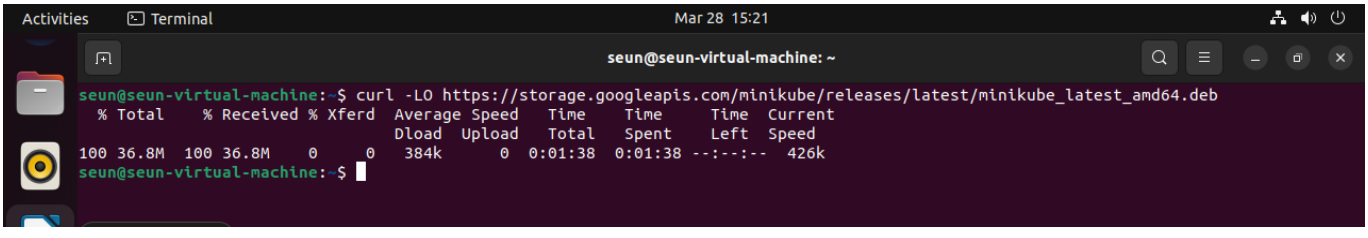
```
newgrp docker
```



◇ PHASE 3: Install Minikube

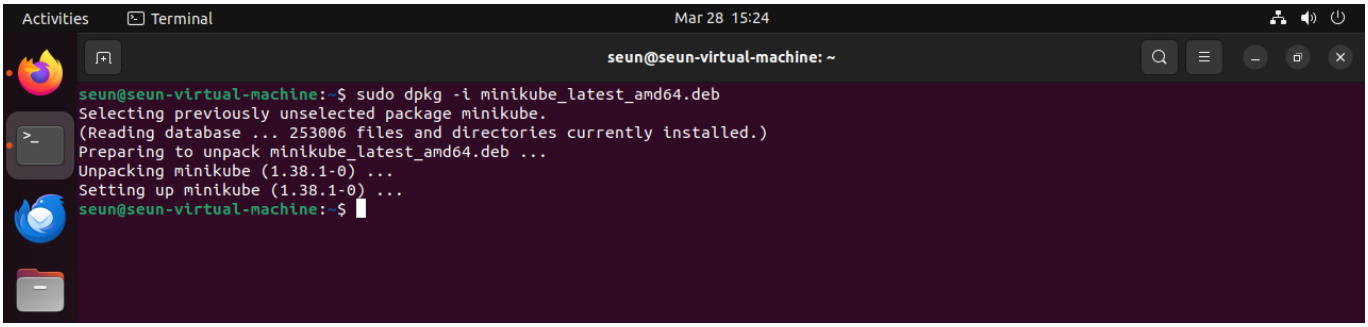
Step 1: Download Minikube

```
curl -LO
https://storage.googleapis.com/minikube/releases/latest/minikube_latest_amd64.deb
```



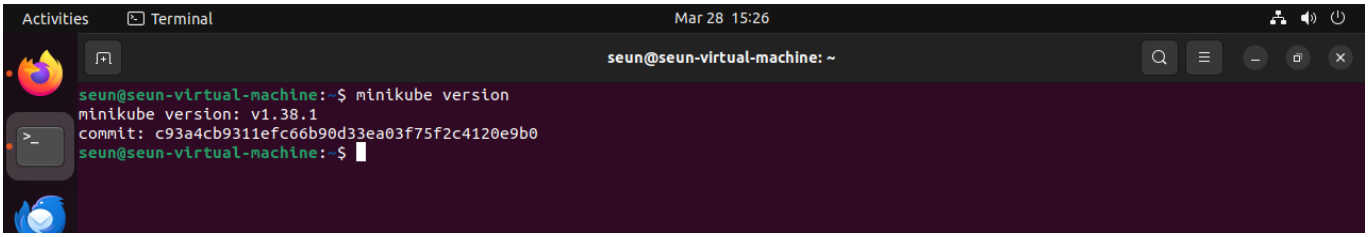
Step 2: Install Minikube

```
sudo dpkg -i minikube_latest_amd64.deb
```



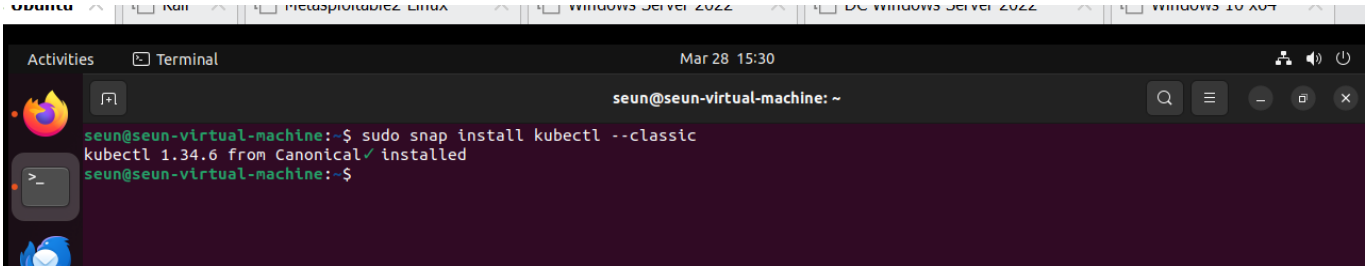
Step 3: Verify installation

```
minikube version
```



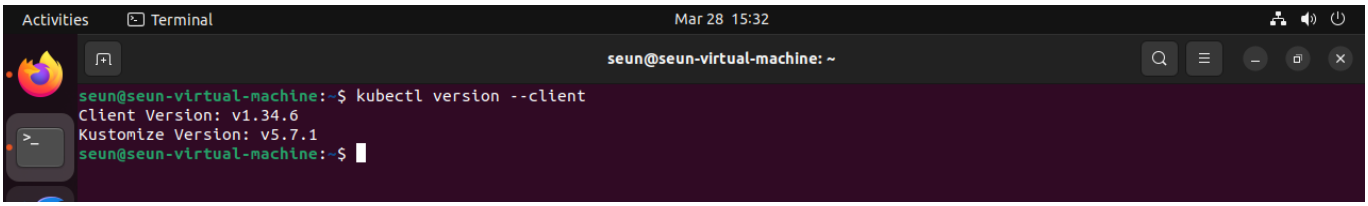
◇ PHASE 4: Install kubectl

```
sudo snap install kubectl --classic
```



Verify kubectl

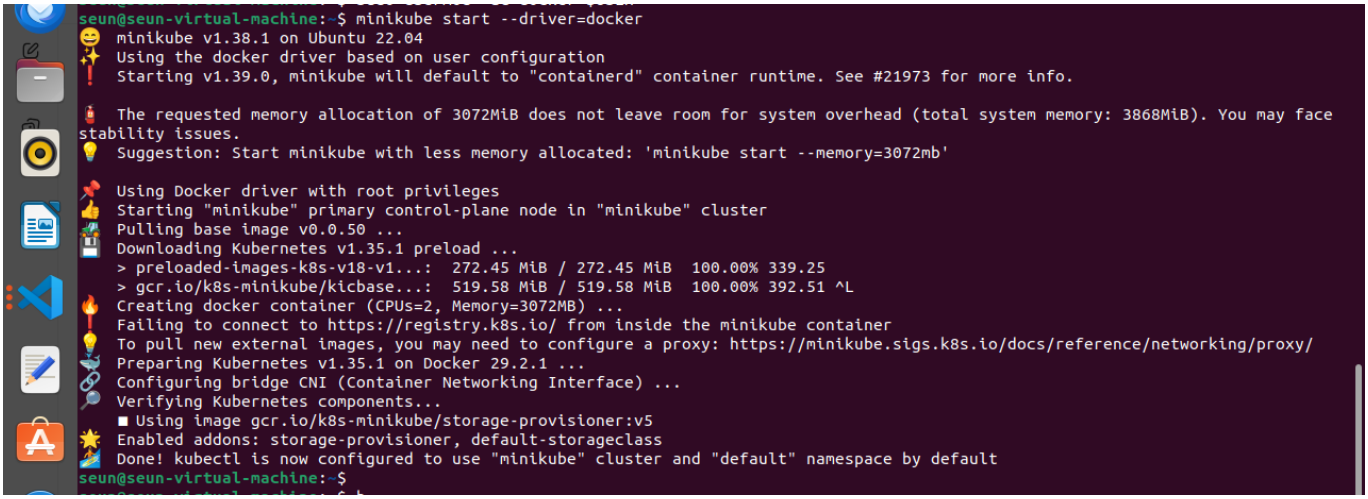
```
kubectl version --client
```



◇ PHASE 5: Start Kubernetes Cluster

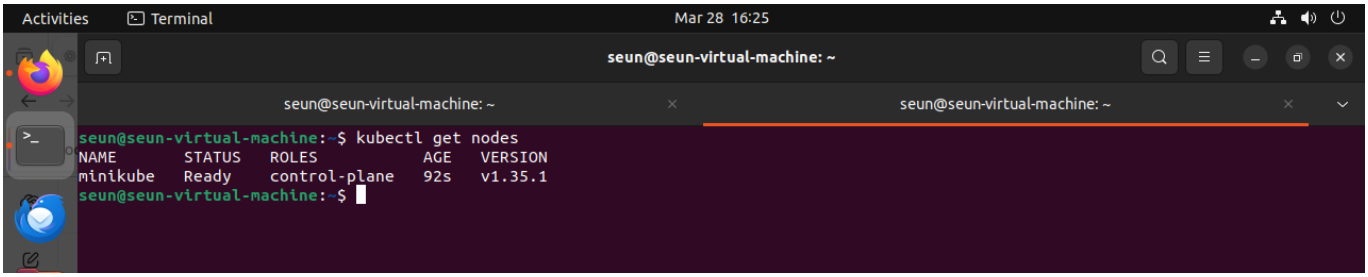
Step 1: Start Minikube

```
minikube start --driver=docker
```



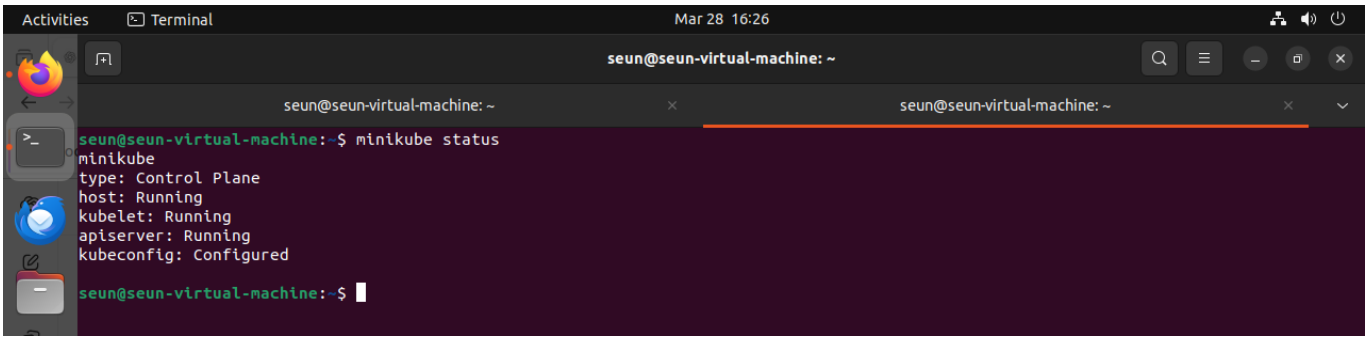
Step 2: Verify node

```
kubectl get nodes
```



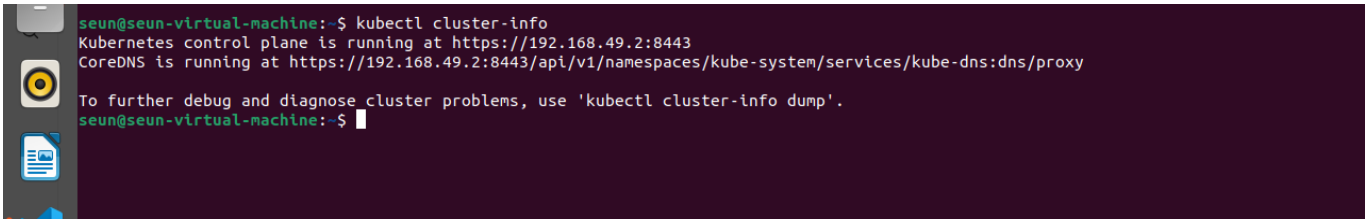
Step 3: Check Minikube status

```
minikube status
```



Step 4: Cluster info

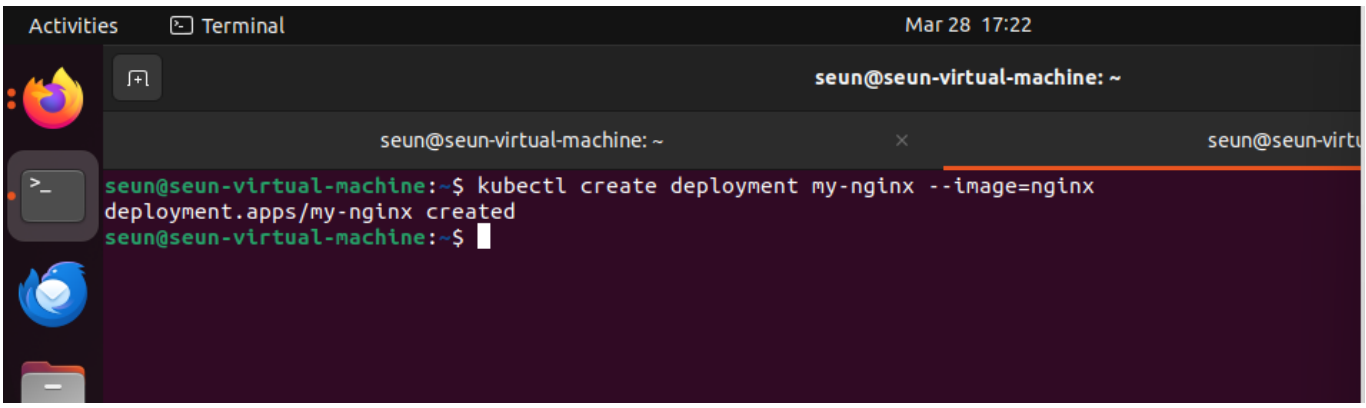
```
kubectl cluster-info
```



◇ PHASE 6: Deploy Application

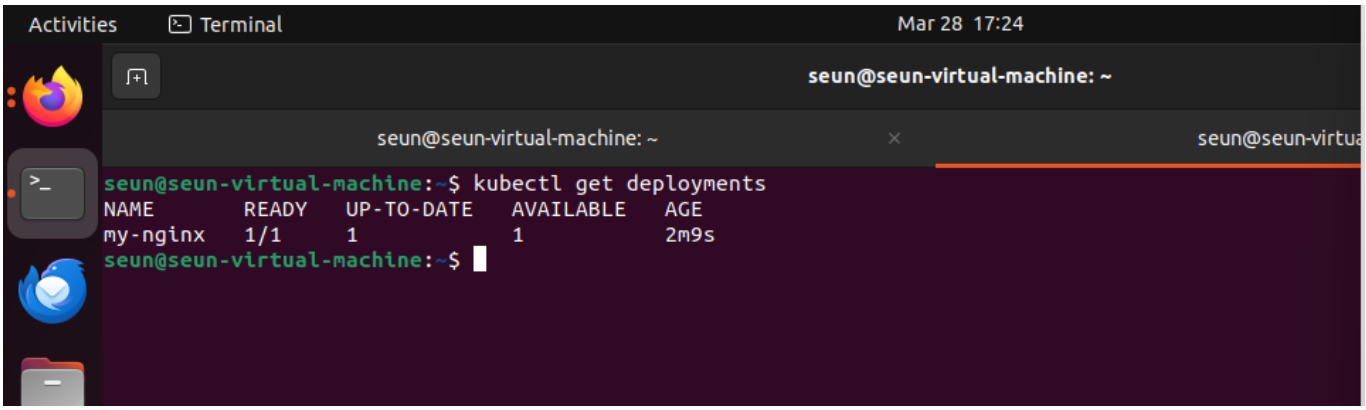
Step 1: Create deployment

```
kubectl create deployment my-nginx --image=nginx
```



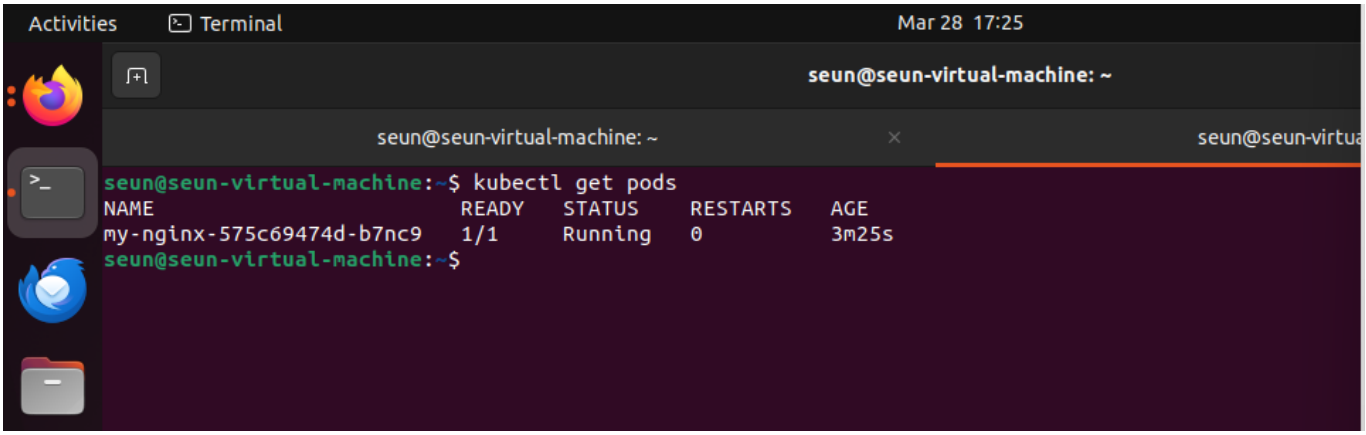
Step 2: Verify deployment

```
kubectl get deployments
```



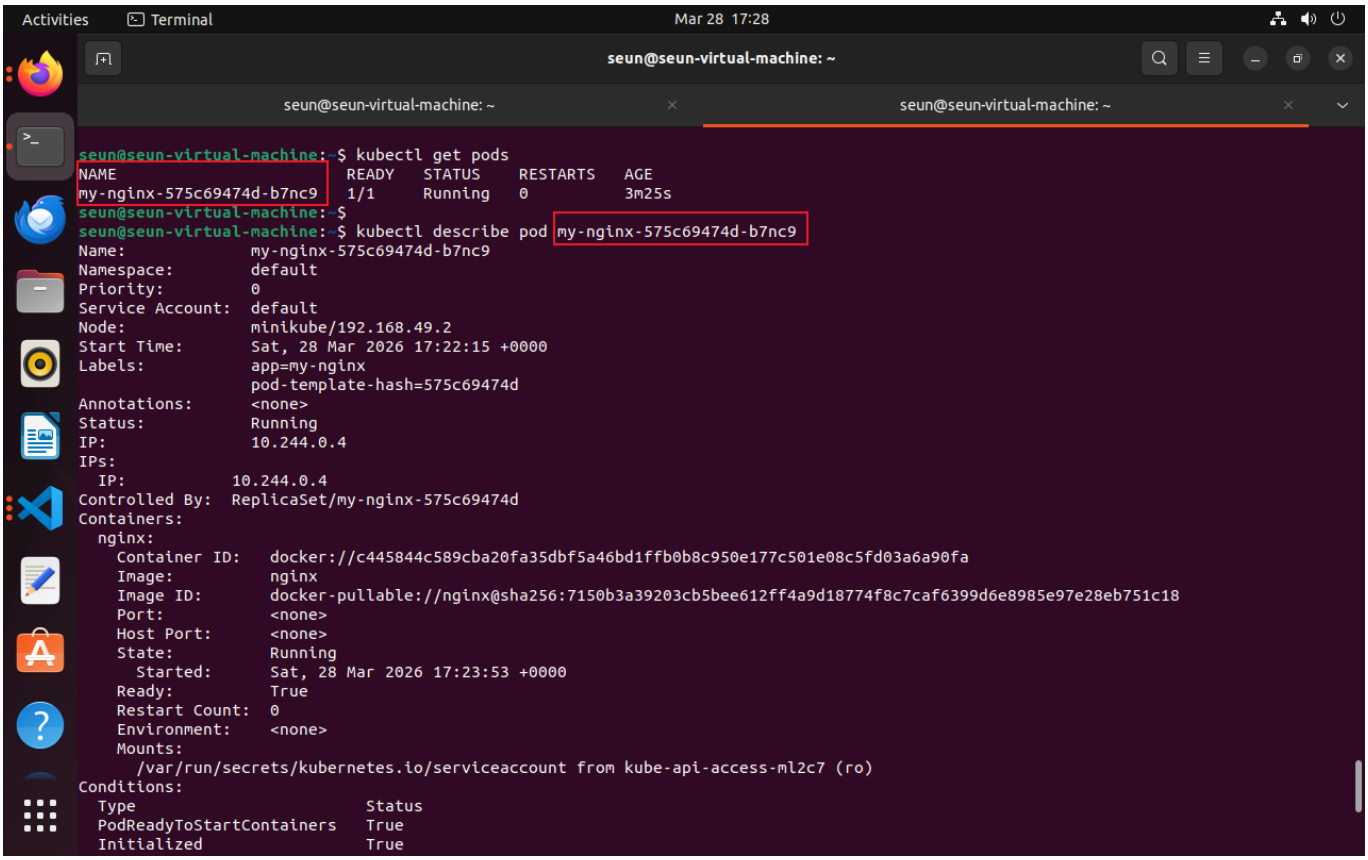
Step 3: Check pods

```
kubectl get pods
```



Step 4: Describe pod

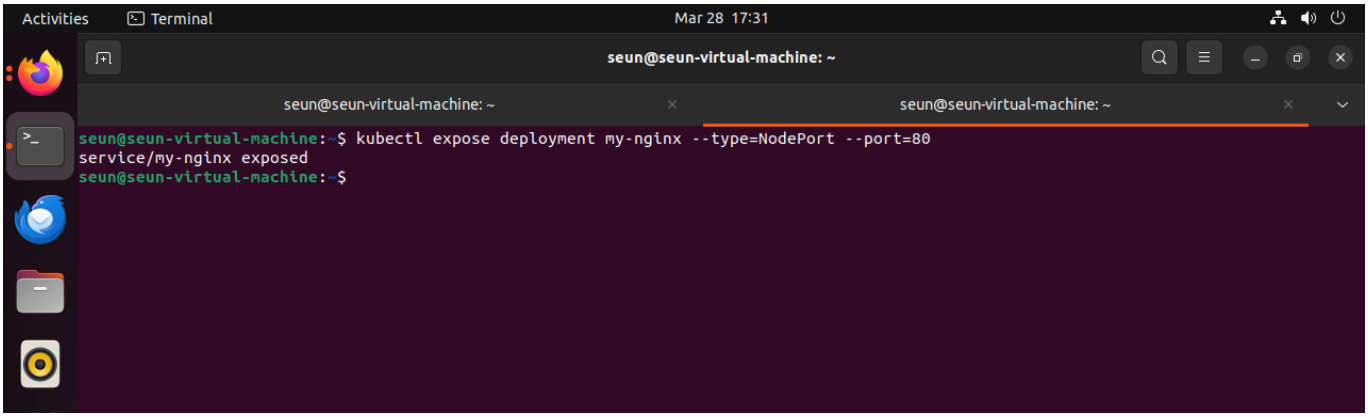
```
kubectl describe pod <pod-name>
```



◇ PHASE 7: Expose Application

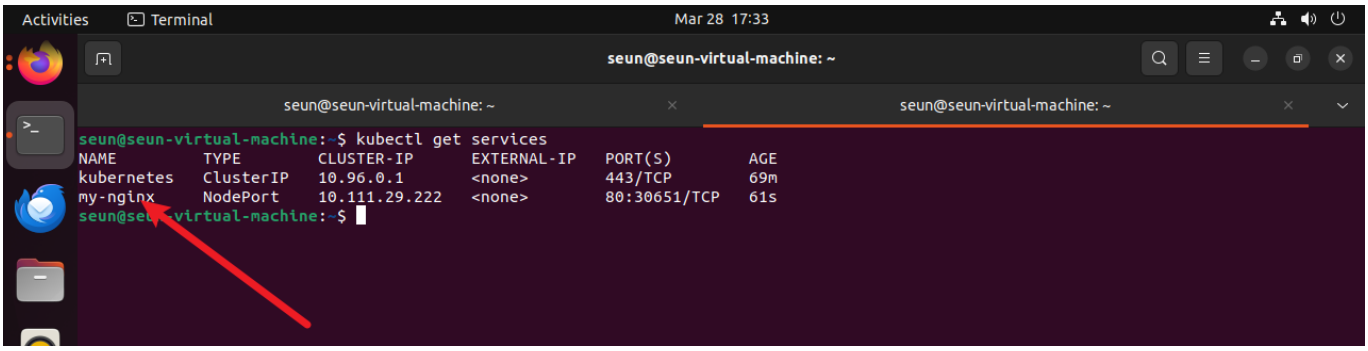
Step 1: Expose deployment

```
kubectl expose deployment my-nginx --type=NodePort --port=80
```



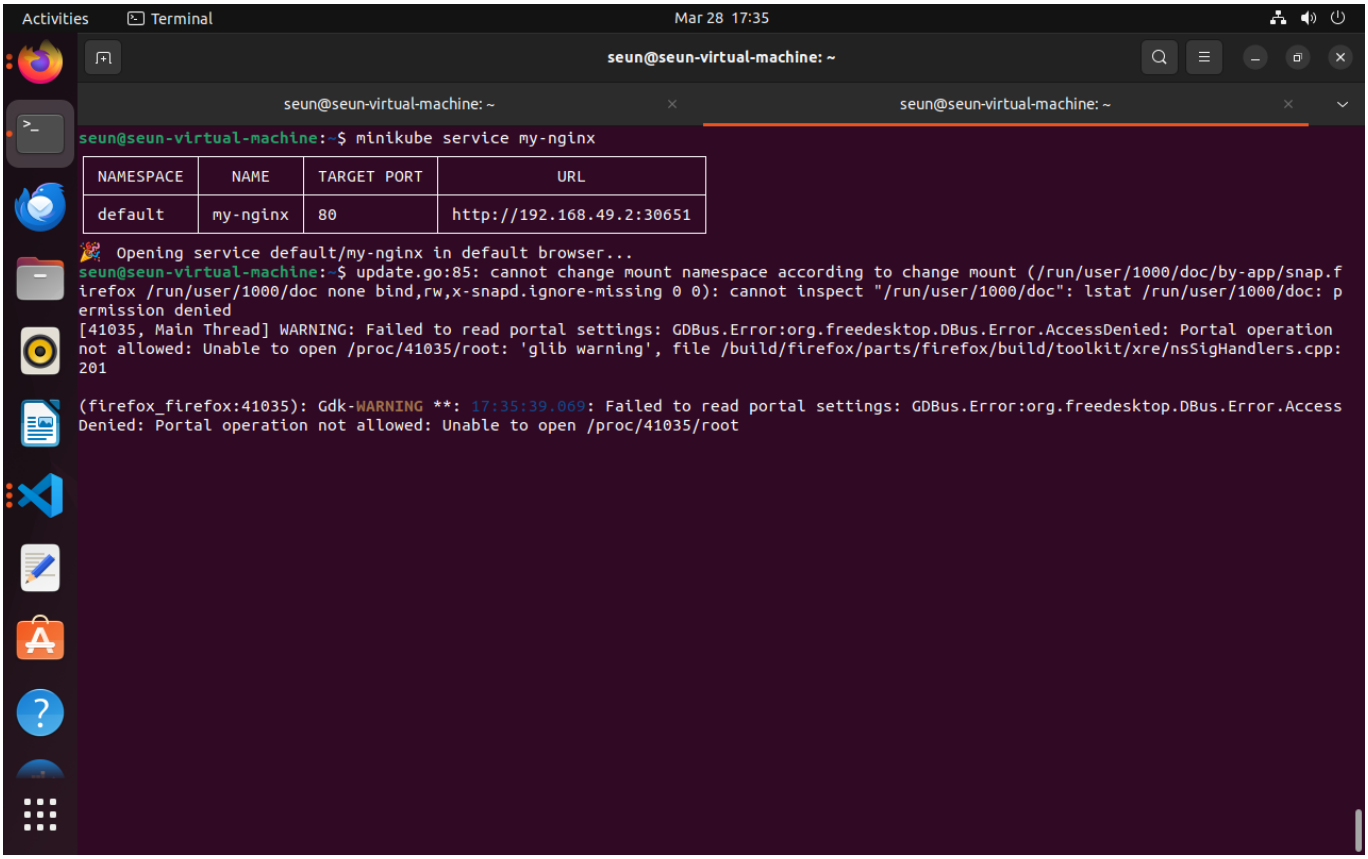
Step 2: Verify service

```
kubectl get services
```

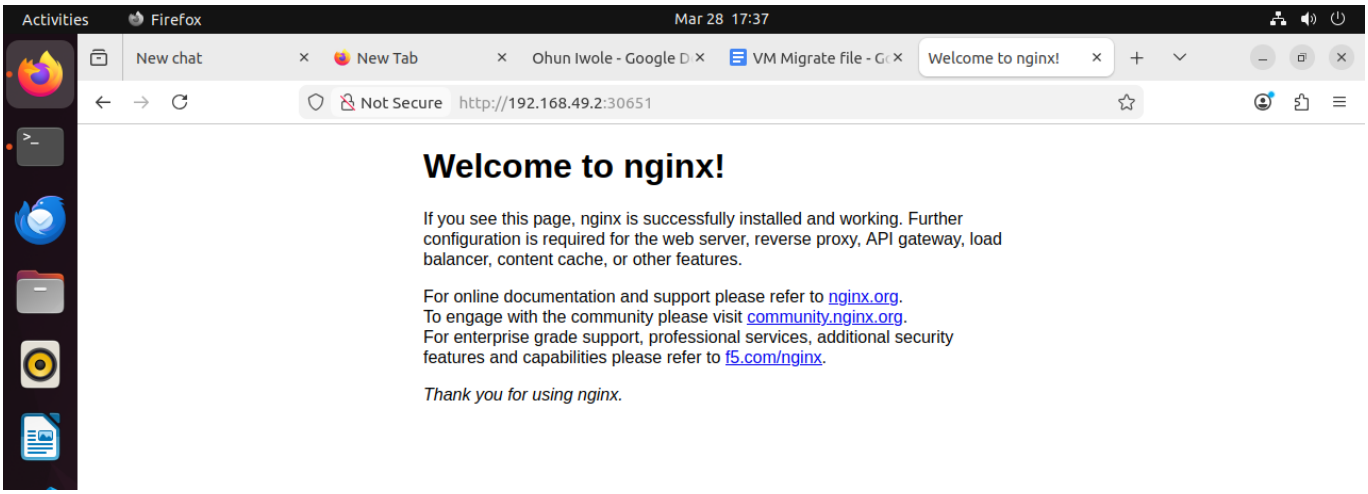


Step 3: Access application

```
minikube service my-nginx
```



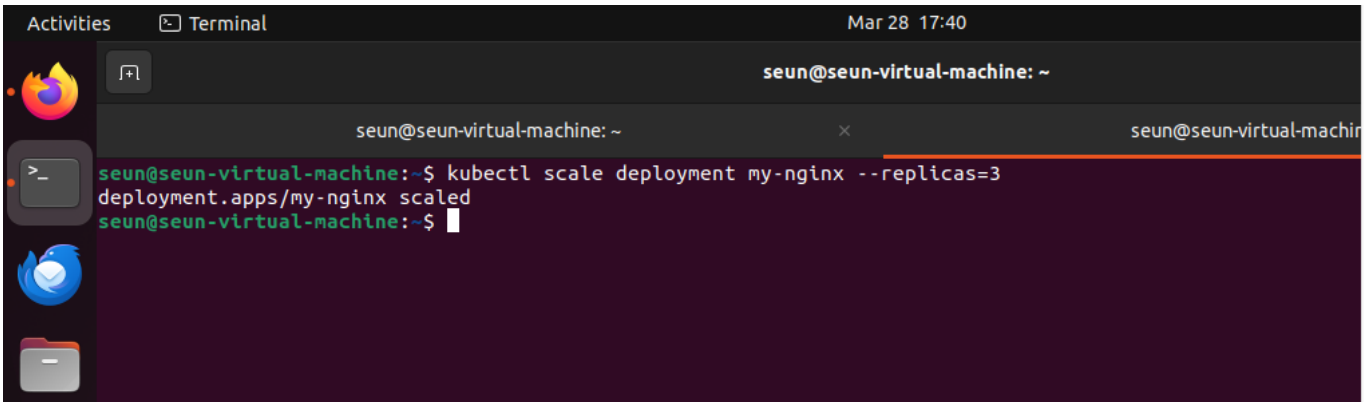
Step 4: Application in browser



◇ PHASE 8: Scaling & Self-Healing

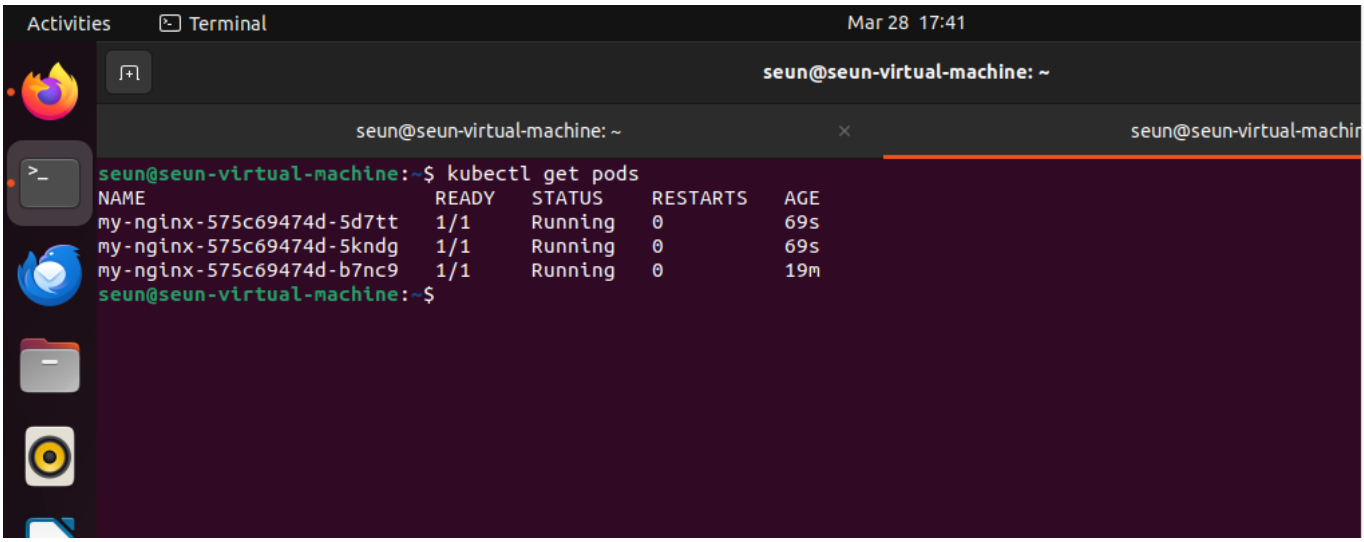
Step 1: Scale application

```
kubectl scale deployment my-nginx --replicas=3
```



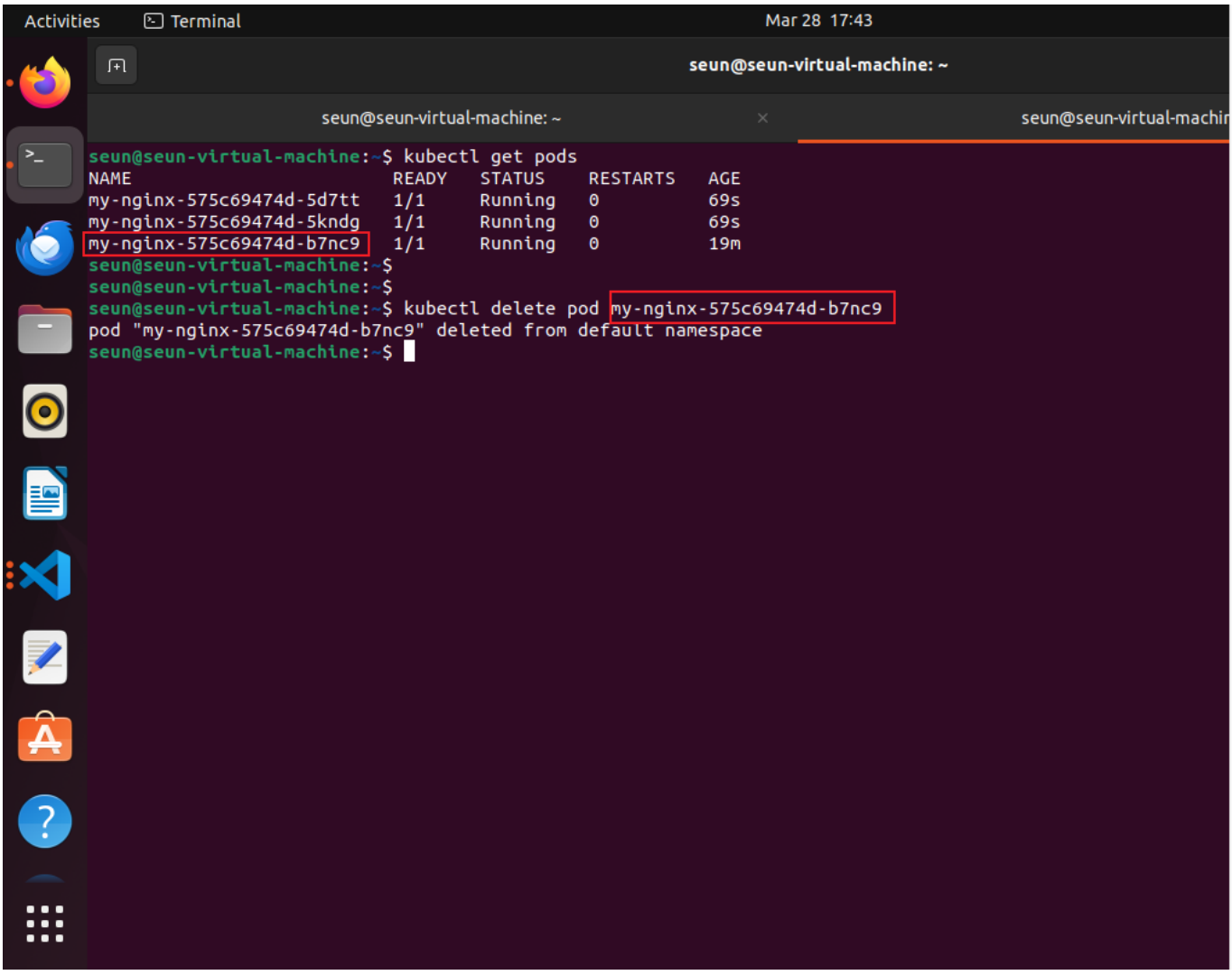
Step 2: Verify pods

```
kubectl get pods
```



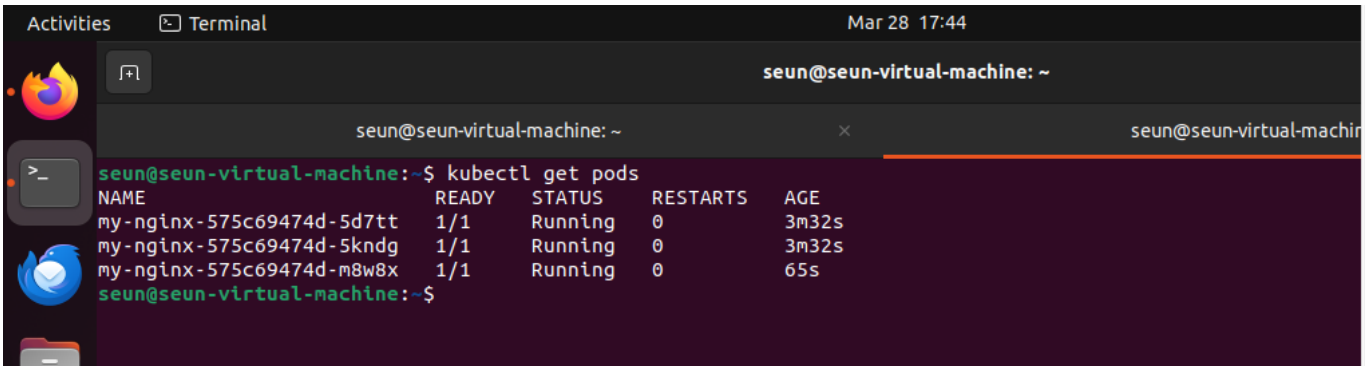
Step 3: Delete a pod

```
kubectl delete pod <pod-name>
```



Step 4: Observe self-healing

```
kubectl get pods
```



🧠 Key Concepts Demonstrated

- Containerization using Docker
- Kubernetes cluster setup with Minikube
- Deployment and management of Pods
- Service exposure using NodePort
- Horizontal scaling
- Self-healing (automatic pod recreation)

🏁 Conclusion

This project successfully demonstrates how Kubernetes orchestrates containerized applications by automating deployment, scaling, and recovery. Using Minikube enabled a local simulation of a production-like Kubernetes environment.